LAPORAN PRAKTIKUM PEKAN 1

STRUKTUR DATA

ABSTRACT DATA TYPE JAM

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Alvero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, Maret 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 2

1.3 Manfaat Praktikum 2

BAB II PEMBAHASAN 3

2.1 Jam_2511531018 3

2.1.1 Kode Program 3

2.1.2 Langkah Kerja 3

2.2 JamDriver_2511531018 5

2.2.1. Kode Program 5

2.2.2. Langkah Kerja 5

2.2.3. Output 6

2.3 JamDriver2_2511531018 7

2.3.1 Kode Program 7

2.3.2 Langkah Kerja 7

2.3.3. Output 9

KESIMPULAN 10

DAFTAR PUSTAKA 11

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data yang berkaitan dengan waktu merupakan salah satu kebutuhan yang cukup penting, terutama pada aplikasi seperti sistem penjadwalan, absensi, pengingat, hingga perhitungan durasi suatu kegiatan. Waktu umumnya direpresentasikan dalam bentuk jam, menit, dan detik yang memiliki batasan nilai tertentu, yaitu jam antara 0 hingga 23, serta menit dan detik antara 0 hingga 59. Jika pengolahan data waktu dilakukan secara manual menggunakan variabel biasa tanpa adanya struktur yang jelas, maka hal tersebut dapat menimbulkan berbagai permasalahan, seperti kesalahan dalam validasi data, kesulitan dalam melakukan operasi perhitungan waktu, serta kurang efisien dalam pengelolaan kode program.

Seiring dengan berkembangnya kebutuhan akan pengolahan data yang lebih kompleks dan terstruktur, diperlukan suatu pendekatan yang mampu mengorganisasi data dan fungsi secara lebih baik, yaitu dengan menggunakan konsep pemrograman berorientasi objek (Object-Oriented Programming/OOP). Dengan pendekatan ini, data dan operasi yang berkaitan dapat dikemas dalam satu kesatuan yang disebut class. Melalui class, pengolahan waktu dapat dilakukan dengan lebih sistematis, karena setiap atribut dan method yang berkaitan dengan waktu berada dalam satu struktur yang terorganisir.

Berdasarkan hal tersebut, dibuatlah sebuah class yang merepresentasikan waktu dalam bentuk jam, menit, dan detik. Class ini tidak hanya berfungsi untuk menyimpan data waktu, tetapi juga menyediakan berbagai method untuk melakukan validasi, konversi waktu ke dalam satuan detik, serta operasi aritmatika seperti penjumlahan dan pengurangan waktu. Selain itu, class ini juga dilengkapi dengan fungsi untuk membandingkan dua waktu dan menampilkan waktu dalam format yang mudah dibaca. Dengan adanya class ini, diharapkan proses pengolahan data waktu menjadi lebih mudah, akurat, dan

efisien, serta dapat digunakan kembali dalam berbagai kebutuhan pemrograman lainnya.

1.2 Tujuan

1.2.1 Merepresentasikan waktu (jam, menit, detik) dalam bentuk class yang terstruktur.

1.2.2 Memastikan nilai waktu valid sesuai batas (jam 0–23, menit dan detik 0–59).

1.2.3 Menyediakan fungsi untuk konversi waktu ke detik dan sebaliknya.

1.2.4 Memudahkan operasi aritmatika waktu seperti penjumlahan dan pengurangan waktu.

1.2.5 Menyediakan fungsi untuk membandingkan dua waktu.

1.2.6 Menampilkan waktu dalam format yang rapi dan mudah dibaca.

1.3 Manfaat Praktikum

1.3.1 Mempermudah pengolahan data waktu dalam berbagai aplikasi.

1.3.2 Mengurangi kesalahan input dengan adanya validasi waktu.

1.3.3 Mempercepat proses perhitungan waktu seperti durasi atau selisih waktu.

1.3.4 Dapat digunakan kembali (reusable) dalam program lain tanpa membuat ulang kode.

1.3.5 Membantu memahami konsep pemrograman berorientasi objek (OOP).

1.3.6 Membuat kode program lebih rapi, terstruktur, dan mudah dipelihara.

BAB II PEMBAHASAN

2.1 Jam_2511531018

2.1.1 Kode Program

```java
package pekan1_2511531018;

public final class Jam_2511531018 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // --- Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }
    // --- Konstruktor/Kreator ---
    public Jam_2511531018(int h, int m, int s) {
        this.hh= h; this.mm = m; this.ss = s;
    }
    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator ---
    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m; }
    public void setSecond(int s) {this.ss = s; }

    // --- Konversi ---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511531018 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; // wrap 24 jam
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511531018(h, m, s);
    }
```

```java
    // --- Relasional ---
    public int compareTo(Jam_2511531018 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511531018 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.mm;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    // --- Aritmatika ---
    public Jam_2511531018 plus(Jam_2511531018 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511531018 minus(Jam_2511531018 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
    public Jam_2511531018 nextSeconds() { return fromSeconds(this.toSeconds() + 1); }
    public Jam_2511531018 nextNSeconds (int n) {return fromSeconds(this.toSeconds() + Math.max(0,  n)); }
    public Jam_2511531018 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
    public Jam_2511531018 prevNSeconds(int n) { return fromSeconds (Math.floorMod(this.toSeconds() - Math.max(0,  n), 24*3600)); }

    // --- Durasi (detik) --bisa negatif jika this > other? sesuai specifikasi durasi(JAw, JAkh)
    public static int durasiDetik(Jam_2511531018 jaw, Jam_2511531018 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }
    //overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

Gambar 2.1.1 Kode program Operator Aritmatika.

2.1.2 Langkah Kerja

2.1.2.1 Program membuat class Jam_2511531018 untuk merepresentasikan waktu. Terdapat 3 atribut yaitu hh => jam (0-23), mm => menit (0-59), dan ss => detik (0-59).

```java
public final class Jam_2511531018 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59
```

Gambar 2.1.2.1 Deklarasi *class* dan atribut.

2.1.2.2 Method ini digunakan untuk mengecek apakah input waktu valid. Mengembalikan true jika jam antara 0-23 dan menit & detik antara 0-59. Kalau tidak sesuai maka false.

```
8     // --- Validator ---
9     public static boolean isValid(int h, int m, int s) {
10        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
```

Gambar 2.1.2.2 Validator (Validasi input).

2.1.2.3 Digunakan saat membuat objek waktu. Nilai h, m, s langsung disimpan ke atribut.

```
12      // --- Konstrukto/Kreator ---
13      public Jam_2511531018(int h, int m, int s) {
14          this.hh= h; this.mm = m; this.ss = s;
```

Gambar 2.1.2.3 Konstruktor.

2.1.2.4 Mengambil nilai jam, menit, dan detik dari objek.

```
16      // --- Selektor ---
17      public int getHour() { return hh; }
18      public int getMinute() { return mm; }
19      public int getSecond() { return ss; }
```

Gambar 2.1.2.4 Selektor (Getter).

2.1.2.5 Mengubah nilai jam, menit, detik setelah objek dibuat.

```
21      // --- Mutator ---
22      public void setHour(int h) {this.hh = h; }
23      public void setMinute(int m) {this.mm = m; }
24      public void setSecond(int s) {this.ss = s; }
```

Gambar 2.1.2.5 Mutator (Setter).

2.1.2.6 Mengubah waktu menjadi total detik dengan rumus jam * 3600, menit * 60, dan detik langsung,

```
26      // --- Konversi ---
27      public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
```

Gambar 2.1.2.4 Konversi ke detik.

2.1.2.7 Jika detik negatif maka error, jika lebih dari 24 jam maka diputar (modulo 24 jam). Konversi jam = total/3600, sisa => menit, dan sisa => detik. Buat objek bari dari hasil tersebut.

```
28      public static Jam_2511531018 fromSeconds(int total) {...
```

Gambar 2.1.2.7 Membuat waktu dari detik.

2.1.2.8 Membandingkan dua waktu berdasarkan total detik. Hasil negatif => lebih kecil, 0 => sama, dan positif => lebih besar. Mengecek apakah dua objek waktu sama persis.

```java
    // --- Relasional ---
    public int compareTo(Jam_2511531018 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511531018 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.mm;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }
```

Gambar 2.1.2.8 Relasional (Perbandingan).

2.1.2.9 Menjumlahkan dua waktu, mengurangkan waktu, memakai floormod agar tidak negatif (berputar 24 jam), menambah detik, dan mengurangi detik.

```java
    // --- Aritmatika ---
    public Jam_2511531018 plus(Jam_2511531018 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511531018 minus(Jam_2511531018 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
    public Jam_2511531018 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
    public Jam_2511531018 nextNSeconds (int n) {return fromSeconds(this.toSeconds() + Math.max(0,  n)); }
    public Jam_2511531018 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
    public Jam_2511531018 prevNSeconds(int n) { return fromSeconds (Math.floorMod(this.toSeconds() - Math.max(0,  n), 24*3600)); }
```

Gambar 2.1.2.9 Operasi aritmatika waktu.

2.1.2.10 Menghitung selisih waktu dalam detik: waktu akhir – waktu awal.

```java
    // --- Durasi (detik) --bisa negatif jika this > other? sesuai spesifikasi durasi(JAw, JAkh)
    public static int durasiDetik(Jam_2511531018 jaw, Jam_2511531018 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }
```

Gambar 2.1.2.10 Durasi (selisih waktu).

2.1.2.11 Mengubah waktu menjadi format dan selalu 2 digit agar rapi.

```java
    //overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

Gambar 2.1.2.11 Output String.

2.2 JamDriver_2511531018

2.2.1. Kode Program

```java
package pekan1_2511531018;

public class JamDriver_2511531018 {
    public static void main(String[] args) {
        Jam_2511531018 a = new Jam_2511531018(23, 59, 50);
        Jam_2511531018 b = new Jam_2511531018(0, 0, 15);
        System.out.println("a              = " + a);
        System.out.println("b              = " + b);
        System.out.println("a+b            = " + a.plus(b));
        System.out.println("next 20s       = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b)    = " + Jam_2511531018.durasiDetik(a,b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
```

Gambar 2.2.1 Kode program JamDriver_2511531018.

2.2.2. Langkah Kerja

2.2.2.1 Program utama dimulai dari main dan semua objek dibuat dan dijalankan.

```java
public class JamDriver_2511531018 {
    public static void main(String[] args) {
```

Gambar 2.2.2.1 Deklarasi class dan main method.

2.2.2.2 Membuat 2 objek waktu: a => jam 23:59:50 dan b => jam 00:00:15 artinya a hampir tengah malam dan b sudah lewat tengah malam.

```
Jam_2511531018 a = new Jam_2511531018(23, 59, 50);
Jam_2511531018 b = new Jam_2511531018(0, 0, 15);
```

Gambar 2.2.2.2 Membuat objek waktu.

2.2.2.3 Memanggil method toString().

```
System.out.println("a                    = " + a);
System.out.println("b                    = " + b);
```

Gambar 2.2.2.3 Menampilkan Nilai Waktu.

2.2.2.4 Mengubah a dan b ke detik, menjumlahkan, dan dikonversi kembali ke format jam.

```
9        System.out.println("a+b                  = " + a.plus(b));
```

Gambar 2.2.2.4 Penjumlahan waktu.

2.2.2.5 Menambah 20 detik ke waktu a.

```
10       System.out.println("next 20s             = " + a.nextNSeconds(20));
```

Gambar 2.2.2.5 Menambah 20 detik.

2.2.2.6 Menghitung durasiDetik(a,b): Menghitung selisih detik.

```
11       System.out.println("durasiDetik(a,b)     = " + Jam_2511531018.durasiDetik(a,b));
```

Gambar 2.2.2.6 Menghitung durasi.

2.2.2.7 Method compareTo(): Bandingkan a dan b berdasarkan detik.

```
12       System.out.println("a.compareTo(b) = " + a.compareTo(b));
```

Gambar 2.2.2.7 Perbandingan waktu.

2.2.3. Output

```
a              = 23:59:50
b              = 00:00:15
a+b            = 00:00:05
next 20s       = 00:00:10
durasi(a,b)    = -86375
a.compareTo(b) = 1
```

Gambar 2.2.3 Hasil output.

2.3 JamDriver2_2511531018

2.3.1 Kode Program

```java
package pekan1_2511531018;

import java.util.Scanner;;
public class JamDriver2_2511531018 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511531018 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511531018 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)      : " + j1.toString());
        System.out.println("Jam 2 (String)      : " + j2.toString());
        System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());
        System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());

        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status    : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status    : Jam 1 lebih awal (sebelum) Jam 2");
        }else {
            System.out.println("Status    Jam 1 dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)       : " + Jam_2511531018.durasiDetik(j1, j2) + " detik");

        Jam_2511531018 jNext = j1.nextSeconds();
        System.out.println("Jam 1 Detik Berikutnya   : " + jNext);
```

```java
        Jam_2511531018 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya   : " + jPrev);

        // 6. Operasi Penjumlahan Jam
        Jam_2511531018 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2            : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }
        /**
         * Prosedur pembantu untuk melakukan input dan validasi secara berulang
         * sampai user memasukkan angka yang benar (0..23, 0..59).
         */
        private static Jam_2511531018 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)      : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59)    : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59)    : ");
            s = sc.nextInt();

            // Memanggil method static isValid dari kelas Jam
            if (Jam_2511531018.isValid(h, m, s)) {
                return new Jam_2511531018(h, m, s);
            } else {
                System.out.println("[Error] input tidak valid! Silakan ulangi.\n");
            }
```

Gambar 2.3.1 Kode program jaDriver2_2511531018.

2.3.2 Langkah Kerja

2.3.2.1 Program dimulai dari method, membuat objek input, dan menampilkan judul program.

```java
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
```

```
System.out.println("=== Program Driver Objek Jam
===");
```

Gambar 2.3.2.1 Inisialisasi program.

2.3.2.2 Menampilkan label, memanggil fungsi buatJamDariInput, dan menghasilkan objek j1 dan j2.

```
// 1. Input Jam Pertama
System.out.println("\n--- Input Jam 1 ---");
Jam_2511531018 j1 = buatJamDariInput(input);

// 2. Input Jam Kedua
System.out.println("\n--- Input Jam 2 ---");
Jam_2511531018 j2 = buatJamDariInput(input);
```

Gambar 2.3.2.2 Input jam 1 dan jam 2.

2.3.2.3 Memakasi method toString() => format waktu dan toSeconds => konversi ke detik total.

```
System.out.println("\n--- Hasil Operasi ---");
System.out.println("Jam 1 (String)            : " + j1.toString());
System.out.println("Jam 2 (String)            : " + j2.toString());
System.out.println("Jam 1 dalam detik         : " + j1.toSeconds());
System.out.println("Jam 2 dalam detik         : " + j1.toSeconds());
```

Gambar 2.3.2.3 Menampilkan data.

2.3.2.4 Membandingkan waktu dalam detik.

```
int perbandingan = j1.compareTo(j2);
if (perbandingan > 0) {
    System.out.println("Status                    : Jam 1 lebih lambat (setelah) Jam 2");
} else if (perbandingan < 0) {
    System.out.println("Status                    : Jam 1 lebih awal (sebelum) Jam 2");
}else {
    System.out.println("Status                    : Jam 1 dan Jam 2 sama persis");
}
```

Gambar 2.3.2.4 Perbandingan waktu.

2.3.2.5 Menghitung selisih waktu dalam detik, menambah 1 detik dari j1, mengurangi 1 detik, dan menjumlahkan dua waktu.

```
System.out.println("Durasi (J1 ke J2)          : " + Jam_2511531018.durasiDetik(j1, j2) + " detik");

Jam_2511531018 jNext = j1.nextSeconds();
System.out.println("Jam 1 Detik Berikutnya     : " + jNext);

Jam_2511531018 jPrev = j1.prevSecond();
System.out.println("Jam 1 Detik Sebelumnya     : " + jPrev);

// 6. Operasi Penjumlahan Jam
Jam_2511531018 jHasilPlus = j1.plus(j2);
System.out.println("Hasil J1 + J2              : " + jHasilPlus);
```

Gambar 2.3.2.5 Menghitung waktu.

2.3.2.6 Menutup scanner dan menampilkan output.

```
input.close();
System.out.println("\nProgram Selesai.");
```

Gambar 2.3.2.6 Penutup.

2.3.2.7 Loop terus sampai valid dan ambil input user.

```java
private static Jam_2511531018 buatJamDariInput(Scanner sc) {
int h, m, s;
while (true) {
    System.out.print("Masukkan Jam (0-23)        : ");
    h = sc.nextInt();
    System.out.print("Masukkan Menit (0-59)      : ");
    m = sc.nextInt();
    System.out.print("Masukkan Detik (0-59)      : ");
    s = sc.nextInt();
```

Gambar 2.3.2.7 Method input dan validasi.

2.3.2.8 Validasi, jika tidak valid program akan kembali ke awal loop.

```java
if (Jam_2511531018.isValid(h, m, s)) {
    return new Jam_2511531018(h, m, s);
} else {
    System.out.println("[Error] input tidak valid! Silakan ulangi.\n");
}
```

Gambar 2.3.2.8 Method input dan validasi.

2.3.3. Output

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)        : 11
Masukkan Menit (0-59)      : 11
Masukkan Detik (0-59)      : 11

--- Input Jam 2 ---
Masukkan Jam (0-23)        : 22
Masukkan Menit (0-59)      : 22
Masukkan Detik (0-59)      : 22

--- Hasil Operasi ---
Jam 1 (String)             : 11:11:11
Jam 2 (String)             : 22:22:22
Jam 1 dalam detik          : 40271
Jam 2 dalam detik          : 40271
Status                     : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2)          : 40271 detik
Jam 1 Detik Berikutnya     : 11:11:12
Jam 1 Detik Sebelumnya     : 11:11:10
Hasil J1 + J2              : 09:33:33

Program Selesai.
```

Gambar 2.3.3 Hasil output.

KESIMPULAN

Berdasarkan program yang telah dibuat, dapat disimpulkan bahwa implementasi class Jam_2511531018, JamDriver_2511531018, dan JamDriver2_2511531018 berhasil digunakan untuk merepresentasikan dan mengelola data waktu dalam bentuk jam, menit, dan detik. Program ini mampu melakukan berbagai operasi terhadap waktu seperti validasi input, konversi ke dalam satuan detik, serta konversi kembali ke format waktu.

Selain itu, program juga dapat melakukan operasi aritmatika waktu seperti penjumlahan, pengurangan, dan penambahan sejumlah detik tertentu dengan memperhatikan batasan 24 jam (*wrap around*). Fitur perbandingan waktu dan perhitungan durasi juga menunjukkan bahwa waktu dapat diolah secara sistematis dengan memanfaatkan konversi ke detik sebagai dasar perhitungan.

Dengan adanya driver program, seluruh fungsi dalam class dapat diuji dan ditampilkan hasilnya, sehingga membuktikan bahwa program berjalan sesuai dengan logika yang telah dirancang. Secara keseluruhan, program ini memberikan pemahaman yang baik tentang konsep pemrograman berorientasi objek, khususnya dalam pengelolaan data waktu dan operasi-operasi yang terkait.

DAFTAR PUSTAKA

[1] Herbert Schildt, *Java: The Complete Reference*, 11th ed. New York, NY, USA: McGraw-Hill Education, 2018.

[2] Oracle Corporation, "Java Platform, Standard Edition Documentation," [Online]. Available: https://docs.oracle.com/javase/8/docs/. [Accessed: Mar. 31, 2026].

[3] Y. Daniel Liang, *Introduction to Java Programming and Data Structures*, 12th ed. Harlow, UK: Pearson Education, 2020.

[4] Joshua Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.